

Representação da Heap em Vetor

Este material aborda como uma Heap é armazenada em memória utilizando vetores, conceito essencial para compreender as operações da estrutura de dados.

1. Por que utilizar um vetor?

Embora a Heap seja desenhada como uma árvore binária, sua implementação normalmente utiliza um vetor. Como a Heap é uma árvore binária completa, todos os seus elementos podem ser armazenados sequencialmente, dispensando ponteiros entre os nós.

2. Árvore x Vetor

```
    50
   /  \
  30   40
 / \   /
10 20 35
```

Vetor: [50, 30, 40, 10, 20, 35]

3. Organização dos índices

Cada posição do vetor representa um nó da árvore. As relações são obtidas apenas por cálculos:

$\text{Pai} = (i - 1) // 2$

$\text{Filho esquerdo} = 2*i + 1$

$\text{Filho direito} = 2*i + 2$

4. Exemplo

Para o elemento de índice 1 (valor 30):

Pai → índice 0 → 50

Filho esquerdo → índice 3 → 10

Filho direito → índice 4 → 20

5. Tabela de correspondência

Índice	Valor	Pai	Filho Esq.	Filho Dir.
0	50	-	30	40
1	30	50	10	20
2	40	50	35	-
3	10	30	-	-
4	20	30	-	-

5	35	40	-	-
---	----	----	---	---

6. Vantagens da representação em vetor

- Menor consumo de memória (não utiliza ponteiros).
- Acesso direto aos elementos por índice.
- Implementação simples e eficiente.
- Facilita operações como inserção, remoção e Heap Sort.

7. Relação com Min Heap e Max Heap

A representação em vetor é exatamente a mesma nas duas estruturas. A diferença está apenas na propriedade da Heap: • Max Heap: o pai possui valor maior ou igual ao dos filhos. • Min Heap: o pai possui valor menor ou igual ao dos filhos.

8. Exemplo em Python

```
heap = [50, 30, 40, 10, 20, 35]

i = 1

pai = (i - 1) // 2
esquerdo = 2 * i + 1
direito = 2 * i + 2

print("Pai:", heap[pai])
print("Filho esquerdo:", heap[esquerdo])
print("Filho direito:", heap[direito])
```

9. Curiosidades

• A Heap continua sendo uma árvore lógica; apenas sua implementação utiliza um vetor. • A raiz sempre ocupa a posição 0 do vetor. • As fórmulas dos índices só funcionam porque a Heap é uma árvore binária completa. • Essa representação reduz a complexidade da implementação das operações.

10. Possíveis perguntas

- Por que a Heap usa vetor em vez de ponteiros?
- Por que as fórmulas de índices funcionam?
- Qual a diferença entre a representação e a propriedade da Heap?
- Qual a vantagem dessa implementação para o Heap Sort?